

## Lesson 4 — Classification in Scikit-learn

---

So far:

Lesson 1: What is Machine Learning? Lesson 2: Linear Regression Lesson 3: Evaluating Regression Models

📖 Lesson 4: Classification

This is an important step because now we'll move from predicting numbers to predicting categories.

### 1. What is Classification?

---

In regression, the output is a number.

Example:

```
House features
  ↓
ML Model
  ↓
₹50 lakh
```

In classification, the output is a category/class.

For example:

```
Email
  ↓
ML Model
  ↓
Spam / Not Spam
```

Other examples:

| Problem           | Output              |
|-------------------|---------------------|
| Spam detection    | Spam / Not Spam     |
| Fraud detection   | Fraud / Not Fraud   |
| Customer churn    | Churn / No Churn    |
| Disease detection | Positive / Negative |
| Image recognition | Cat / Dog           |
| Loan approval     | Approved / Rejected |

### 2. Regression vs Classification

---

This distinction is extremely important.

#### Regression

Predicts a continuous numerical value.

```
Input
↓
Model
↓
75.4
```

Example:

```
House → ₹75.4 lakh
```

## Classification

Predicts a class/category.

```
Input
↓
Model
↓
1
```

Where:

```
1 = Fraud
0 = Not Fraud
```

So:

```
Regression → number
Classification → category
```

## 3. What is a Class?

---

A class is simply a possible category.

Suppose we want to detect whether a customer will leave a company.

We could define:

```
0 → Customer will stay
1 → Customer will leave
```

Here:

```
0 and 1
```

are our classes.

This is called binary classification because there are only two classes.

## 4. Binary Classification

---

Binary = two

Examples:

```
Spam → 1
Not Spam → 0
```

```
Fraud      → 1
Not Fraud  → 0
```

```
Churn      → 1
No Churn   → 0
```

The model learns:

```
X → y
```

For example:

```
X = customer information

Age
Monthly charges
Tenure
Number of complaints
...
```

and:

```
y = 0 or 1
```

## 5. Multiclass Classification

---

Sometimes there are more than two classes.

For example, suppose we classify animals:

```
0 → Cat
1 → Dog
2 → Horse
```

Now there are 3 classes.

This is called multiclass classification.

Another example:

```
0 → Low
1 → Medium
2 → High
```

## 6. Our First Classification Model

---

We'll start with one of the most important algorithms:

Logistic Regression

Despite having "Regression" in its name, `LogisticRegression` is primarily used for classification.

Scikit-learn:

```
from sklearn.linear_model import LogisticRegression
```

Create the model:

```
model = LogisticRegression()
```

Train:

```
model.fit(X_train, y_train)
```

Predict:

```
predictions = model.predict(X_test)
```

Very similar to what you already learned:

```
model.fit(...)
model.predict(...)
```

## 7. Let's Use a Real Dataset

---

Scikit-learn provides a famous dataset called Breast Cancer Wisconsin.

Don't worry about the name right now—we are using it purely as a machine-learning classification dataset.

It contains numerical measurements and a target representing two classes.

Import it:

```
from sklearn.datasets import load_breast_cancer
```

Load it:

```
data = load_breast_cancer()
```

Let's see the features:

```
print(data.feature_names)
```

And the target:

```
print(data.target)
```

## 8. Understanding X and y Again

---

Remember:

```
X = Features
y = Target
```

For this dataset:

```
X = data.data  
y = data.target
```

So:

```
X
```

contains the input measurements.

And:

```
y
```

contains the class.

You can check:

```
print(X.shape)  
print(y.shape)
```

## 9. Complete First Classification Program

---

Run this in your Jupyter Notebook:

```
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score  
  
# Load dataset  
data = load_breast_cancer()  
  
# Features and target  
X = data.data  
y = data.target  
  
# Split data  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
  
# Create model  
model = LogisticRegression(max_iter=10000)  
  
# Train model  
model.fit(X_train, y_train)  
  
# Make predictions  
predictions = model.predict(X_test)  
  
# Evaluate  
accuracy = accuracy_score(y_test, predictions)  
  
print("Accuracy:", accuracy)
```

You should get an accuracy around 95–97%, although the exact value can depend on the setup/version.

## 10. Let's Understand Every Part

---

### Step 1

```
data = load_breast_cancer()
```

Loads the dataset.

### Step 2

```
X = data.data
```

Stores the features.

Think:

```
X
|
├─ feature 1
├─ feature 2
├─ feature 3
├─ ...
└─ feature 30
```

### Step 3

```
y = data.target
```

Stores the answer/class.

For example:

```
y
0
1
1
0
1
...
```

### Step 4

```
train_test_split(...)
```

Splits the dataset:



### Step 5

```
model = LogisticRegression(max_iter=10000)
```

Creates the classification model.

## Step 6

```
model.fit(X_train, y_train)
```

The model learns patterns from the training data.

## Step 7

```
predictions = model.predict(X_test)
```

The trained model predicts classes for unseen test data.

For example:

Actual:

1 0 1 1 0

Predicted:

1 0 1 0 0

# 11. Accuracy

---

We used:

```
from sklearn.metrics import accuracy_score
```

Then:

```
accuracy_score(y_test, predictions)
```

Accuracy means:

What percentage of predictions were correct?

Suppose:

100 test examples

and the model correctly predicts:

95

Then:

```
Accuracy = 95 / 100  
          = 0.95  
          = 95%
```

# 12. Important Difference From R<sup>2</sup>

---

Earlier you learned:

R<sup>2</sup> → Regression

Now:

```
Accuracy → Classification
```

For example:

### Regression

```
Actual price: 50  
Predicted price: 48
```

```
R2  
MAE  
MSE  
RMSE
```

### Classification

```
Actual: Fraud  
Predicted: Fraud
```

```
Accuracy  
Precision  
Recall  
F1-score
```

We'll learn those classification metrics in detail soon.

## 13. What Does `predict()` Return?

This is important.

With classification:

```
predictions = model.predict(X_test)
```

you get classes.

For example:

```
print(predictions[:10])
```

might produce:

```
[0 1 1 0 1 1 0 0 1 1]
```

These are class predictions.

## 14. `predict()` vs `predict_proba()`

This is a very useful concept.

**`predict()`**

Gives the final class:

```
model.predict(X_test)
```

Example:

```
[0, 1, 1, 0]
```

**predict\_proba()**

Gives probabilities for each class:

```
model.predict_proba(X_test)
```

Example:

```
[[0.92, 0.08],  
 [0.15, 0.85],  
 [0.20, 0.80],  
 [0.73, 0.27]]
```

Meaning for the first example:

```
Class 0 → 92%  
Class 1 → 8%
```

So the model predicts:

```
0
```

because class 0 has the higher probability.

## 15. Why **predict\_proba()** Is Important for SentinelAI

This is actually very relevant to your SentinelAI project.

Your system doesn't just want:

```
Failure
```

It wants something like:

```
Failure Probability: 87%
```

Conceptually:

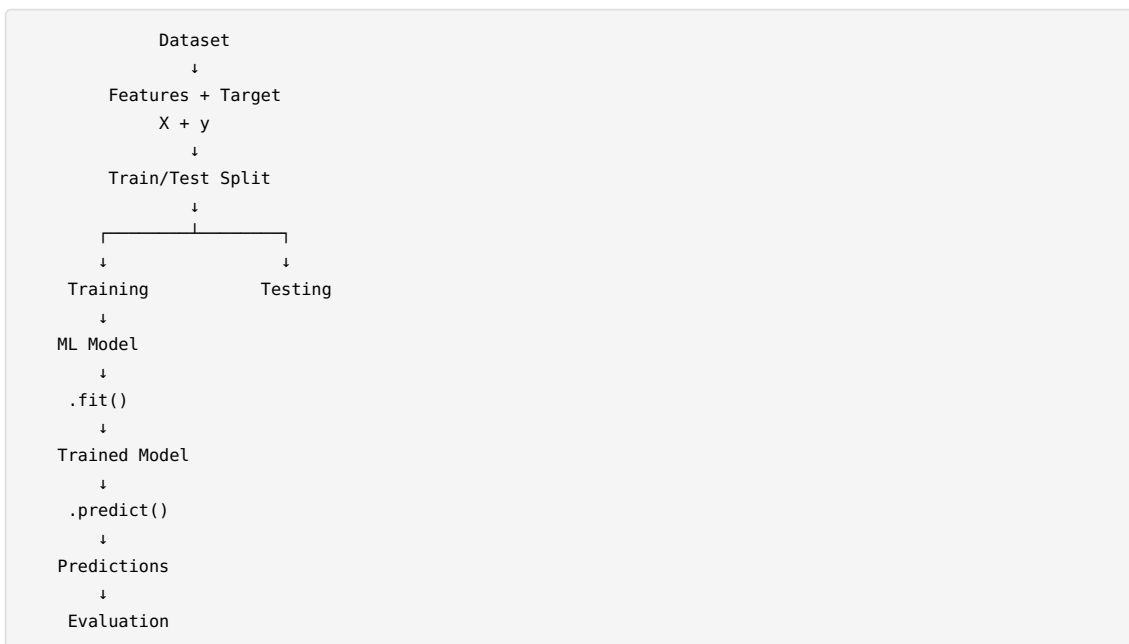
```
CPU  
Memory  
Latency  
Error Rate  
↓  
ML Model  
↓  
Failure Probability  
↓  
87%
```

That's why classification probability concepts will become useful when we build the failure-prediction model later.

## 16. The Classification Workflow

---

You should now recognize this pattern:



For classification:

```
Evaluation
↓
Accuracy
Precision
Recall
F1
Confusion Matrix
```

We'll learn these next.

## 🧠 Lesson 4 — What You Should Remember

---

### 1.

Classification predicts categories.

```
Spam / Not Spam
Fraud / Not Fraud
```

### 2.

Binary classification:

```
2 classes
```

Multiclass classification:

```
3+ classes
```

### 3.

A common Scikit-learn classification model:

```
LogisticRegression()
```

**4.**

Training:

```
model.fit(X_train, y_train)
```

**5.**

Prediction:

```
model.predict(X_test)
```

**6.**

Classification can produce probabilities:

```
model.predict_proba(X_test)
```

**7.**

A common first classification metric:

```
accuracy_score(y_test, predictions)
```

## Exercises

---

Try answering these without looking back:

1. What is the difference between regression and classification?
2. What does binary classification mean?
3. If:

```
0 = Not Fraud  
1 = Fraud
```

what type of problem is this?

4. What does this do?

```
model.fit(X_train, y_train)
```

5. What does this do?

```
model.predict(X_test)
```

6. What's the difference between:

```
predict()
```

and

```
predict_proba()
```

7. If a model gets 90 correct predictions out of 100, what is its accuracy?

Next → Lesson 5: Classification Evaluation

We'll go deep into:

```
graph TD; A[Accuracy] --> B[Confusion Matrix]; B --> C[Precision]; C --> D[Recall]; D --> E[F1 Score]
```

Accuracy  
↓  
Confusion Matrix  
↓  
Precision  
↓  
Recall  
↓  
F1 Score

And I'll explain TP, TN, FP, FN with a simple real-world fraud-detection example, because these are essential for ML interviews and real projects.